TPRE523

Gordillo Loïs, Boufenouche Mehdi, Momo Carnegie & Moragny Alban

Spécification technique

Mise en production d'une solution I.A à partir des éléments de contextes fournis (Cahier des charges - données d'entrée)

20 Mars 2025 - 02 Mai 2025

Table des matières

Table des matières **2**
Introduction **4**
Analyse & Cahier des Charges : Préparation et Collecte des Données 4
- Objectif principal 4
- Contraintes techniques 4
- Livrables attendus 4
- Outils & technologies utilisés 5

Solution & Stack choisis **5**
Back-end Python Django 5
Front-end avec Flutter 5
BDD en PostGreSQL 6
GitLab pour la supervision / le versionning / CI-CD 7
Dockerisation avec Docker 7
Gestion du projet & Gantt 7
Modélisation des données 7
BACK-END avec Django **9**
Objectifs et rôle du back-end 9
Choix techniques et architecture 9
Méthodologie et développement 10
Conteneurisation (Docker) 10
Accessibilité & API publique 11
FRONT-END avec Flutter **12**
1. Objectifs fonctionnels de l'application 12
2. Design de l'application et maquettage initial 12
Objectifs du design initial : 12
Contenu du design : 12
3. Fonctionnalités mises en œuvre 13
3.1. Détection d'empreintes animales (fonction principale) 13
3.2. Gestion des données et persistance 14
3.3. Gestion de la galerie utilisateur 14
3.4. Fiches d'animaux (standard et IA) 14
3.5. Navigation par catégories 15
3.6. Section articles 15
3.7. Page d'accueil dynamique 15
3.8. Gestion des utilisateurs 15
3.9. Comportements intelligents 15
3.10. Paramètres & accessibilité 16
3.11. Widgets personnalisés réutilisables 16
3.12 Gestion des permissions 16
4. Composants techniques et structure fonctionnelle 16

4.1 Architecture logique de l'application....16
4.2 Arborescence technique principale....17
4.3 Pages principales....18
4.4 Services métiers....19
4.5 Modèles utilisés....19
4.6 Widgets personnalisés....19
4.7 Pages authentifications....19
4.8 Navigation et transition entre les pages....19
5. Problèmes rencontrés et solutions appliquées....20
5.1. Problème d'affichage (overflow / SizedBox)....20
5.2. Permissions refusées (caméra, galerie, stockage)....20
5.3. Problème de compatibilité backend (analyse IA – erreur 404)....21
5.4. Problème de chargement d'image d'animaux....21
5.5. Problème de compatibilité multiplateforme (web/desktop)....21
5.6. Problème de cache non persistant ou mal chargé....22
5.7. Problème de navigation et comportement post-analyse IA....22
Mise en oeuvre des plans de tests pour le déploiement....22
Plans de tests fonctionnels et structurels....22
Tests unitaires et d'intégration....23
Tests de sécurité....23
Tests de non-régression....23
Analyse des résultats et corrections....24
Gestion des déséquilibres de classes....24
Détection de fuites de données....24
Supervision de la solution IA via le monitorage....25
Outils de monitorage opérationnels....25
Surveillance des performances....25
Interprétabilité....25
Méthodologie adapté....26
Traçabilité historique via l'export .npz....26
Intégration proactive au CI/CD....26
Visualisation centralisée....26
Détection proactive d'anomalies....27
Pipeline CI/CD comme bouclier préventif....27
Validation matérielle et résilience GPU....27
Correction des dysfonctionnements....27
Stratégies de correction cohérentes....27
Optimisations techniques....28
Amélioration continue et évolutions fonctionnelles....28
Fine-tuning dynamique....28
Mécanismes d'adaptation future....28

Optimisations post-déploiement....28
DÉPLOIEMENT....29
Déploiement de la solution....29
1. Conteneurisation avec Docker....29
2. Intégration Continue (CI) avec GitLab....29
3. Déploiement Continu (CD) avec Railway....29
Avantages de Railway :....30
Perspective :....30
Conclusion:....30
Sources....32
Images :....32
Métadonnées :....32
Autres sources....32
Glossaire....33

Introduction

Le projet **WildLens** vise à développer une solution d'intelligence artificielle capable d'identifier automatiquement des empreintes animales à partir d'images fournies par les utilisateurs. Conçue comme une application complète, la solution repose sur l'intégration d'un modèle de classification, d'une API back-end sécurisée, d'une base de données relationnelle et d'une interface front-end accessible. Cette initiative répond à un besoin croissant d'outils intelligents dans le domaine de la biodiversité et de l'analyse environnementale, en mettant l'accent sur la fiabilité, la performance et la facilité de déploiement. L'ensemble du développement a été pensé pour garantir une portabilité maximale, une automatisation continue des tests, ainsi qu'une traçabilité rigoureuse des résultats.

Analyse & Cahier des Charges : Préparation et Collecte des Données

🔹 **Objectif principal**

Développer une application IA capable de :

- Identifier et classifier automatiquement des empreintes animales,
- Gérer les métadonnées liées aux utilisateurs, aux espèces et aux images,
- Proposer une interface front-end fluide et responsive,
- Garantir la sécurité, la portabilité et l'intégrité des données.

🔹 **Contraintes techniques**

- Portabilité assurée via Docker et Docker Compose,
- Interopérabilité entre les modules IA, back-end, base de données et front-end,
- Automatisation avec des pipelines CI/CD pour tests, sécurité et déploiement, Traçabilité des performances du modèle via export de métriques et suivi historique,
- Conformité RGPD avec anonymisation et stockage contrôlé des données.

🔹 **Livrables attendus**

- API REST documentée via Swagger,
- Interfaces mobiles (Flutter) et web (React),
- Base de données PostgreSQL conteneurisée,
- Modèle IA entraîné, testé, sécurisé et monitoré,

- Pipeline CI/CD intégré (tests unitaires, sécurité, régressions),
- Documentation technique et fonctionnelle, schémas UML et MCD.

🔹 **Outils & technologies utilisés**

- Back-end : Django & Django REST Framework
- Front-end : Flutter / React
- Base de données : PostgreSQL
- Conteneurisation & déploiement : Docker, GitLab CI/CD, Railway
- Suivi & monitoring IA : pytest, Grad-CAM, profils de performance, export .npz, dashboard Grafana (en perspective)

Solution & Stack choisis

Back-end Python Django

Nous avons fait le choix du framework Django, développé en Python, pour concevoir l'API de notre application. Plusieurs raisons motivent cette sélection :

Compatibilité avec l'IA : Django, étant basé sur Python, s'intègre naturellement avec les bibliothèques de machine learning que nous utilisons pour la classification d'empreintes. De même, Django fournit une structure "batteries incluses" qui accélère significativement le développement. Il offre notamment une gestion native de l'ORM, de l'authentification, des migrations et des vues REST via Django REST Framework.

API REST modulaire : Avec Django REST Framework, nous avons pu créer des endpoints clairs, testables, et documentés, facilitant la communication avec le front-end et les systèmes tiers (ex : monitoring, visualisation). Enfin, Django est facilement conteneurisable, ce qui nous permet de répondre aux exigences du cahier des charges quant à la portabilité de la solution.

Front-end avec Flutter

Nous avons comparé plusieurs frameworks avant de choisir Flutter pour des raisons bien précises :

- Performance native : Flutter compile directement en code natif, ce qui permet une fluidité supérieure aux frameworks comme React Native ou Ionic, surtout pour des traitements lourds comme la caméra ou les appels à l'IA.

- Support multiplateforme intégré : Contrairement à React (qui demande React Native pour mobile), Flutter permet de créer une seule base de code qui fonctionne sur Web, Android, iOS et Desktop.
- Interface cohérente sur toutes les plateformes : Flutter n'utilise pas le DOM comme React. Cela rend les transitions, animations et performances graphiques bien plus stables, car tout est dessiné directement via Skia.
- Simplicité de gestion des permissions : Flutter permet d'intégrer facilement des bibliothèques comme permission_handler qui simplifient la gestion multiplateforme des autorisations pour la caméra, le stockage, etc.
- Widgets hautement personnalisables : Les widgets de Flutter nous ont permis de créer des composants graphiques réutilisables pour chaque section de l'application tout en conservant une cohérence parfaite du design.

BDD en PostGreSQL

Pour le stockage des données (utilisateurs, empreintes, animaux identifiés, métadonnées), nous avons retenu PostgreSQL, pour les raisons suivantes :

PostgreSQL excelle dans le traitement de requêtes complexes, indispensable pour filtrer les empreintes, agréger les données par zones ou périodes, ou encore générer des visualisations dynamiques. Cette base est conçue pour supporter des volumes importants, que ce soit pour les images associées aux empreintes ou pour les données enrichies sur les espèces animales.

Fiabilité et écosystème : PostgreSQL est reconnu pour sa stabilité, sa documentation claire, et sa compatibilité avec de nombreux outils que nous avons exploités, comme DBeaver pour la gestion visuelle ou Grafana pour la datavisualisation.

Interopérabilité avec Docker : Son intégration via Docker Compose nous a permis d'assurer une cohérence dans les environnements de développement, de test et de production, tout en facilitant le travail collaboratif.

Le choix de PostgreSQL a été fait après comparaison avec d'autres solutions comme SQLite, MySQL ou MongoDB, qui ne répondaient pas aussi efficacement à nos besoins spécifiques (requêtes analytiques, scalabilité, intégrité des données relationnelles).

GitLab pour la supervision / le versionning / CI-CD

Dockerisation avec Docker

Pour faciliter le déploiement de notre solution, nous avons choisi d'utiliser Docker. La dockerisation permet de créer des conteneurs pour chaque composant de notre stack (Python, PySpark, PostgreSQL, Grafana), assurant ainsi une installation et une mise à jour simplifiées. Cela garantit également la portabilité de notre solution sur différents environnements, que ce soit en développement, en test ou en production (Surtout pour la partie 2 du projet).

Gestion du projet & Gantt

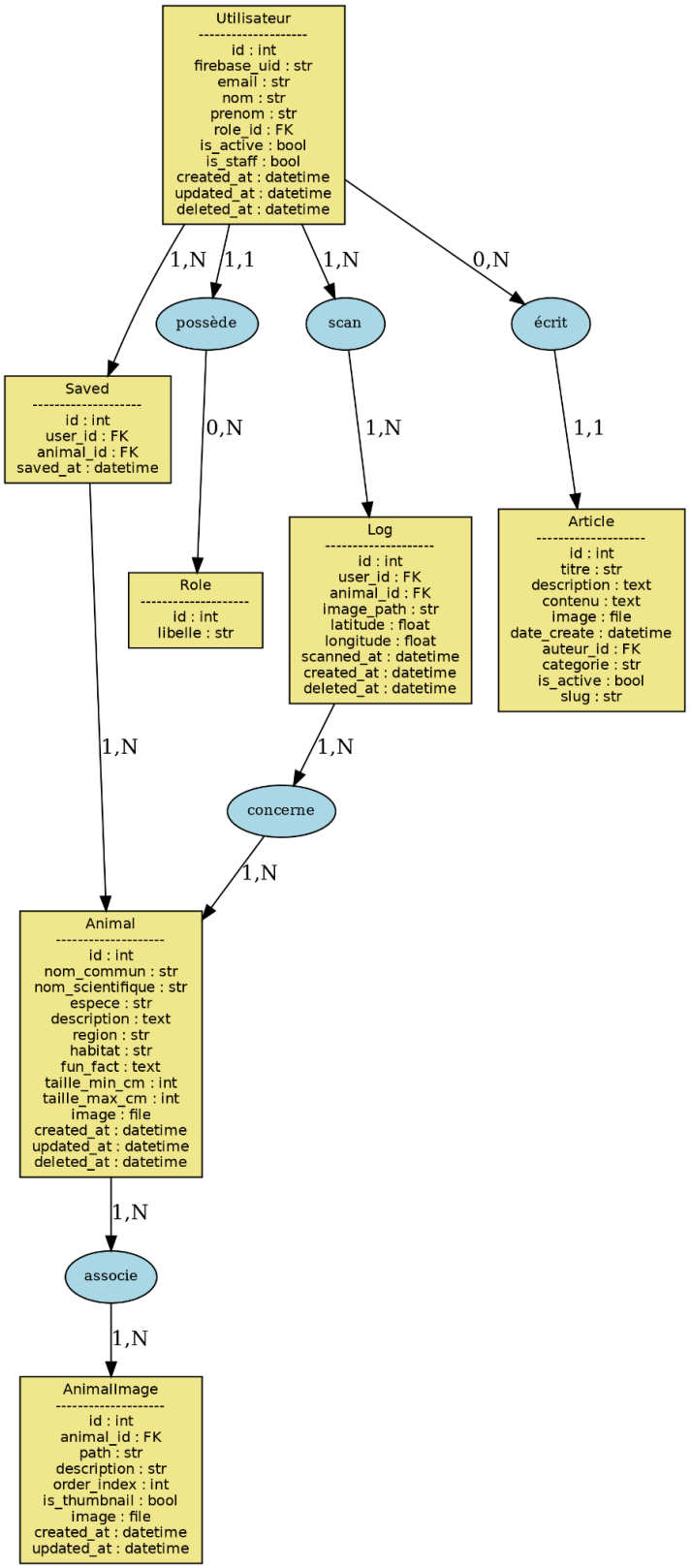
Un diagramme de Gantt a été réalisé pour mieux structurer le projet ainsi que pour suivre l'évolution des tâches. Il a été défini au début du projet mais il a évolué tout au long du projet, au fur et à mesure où nous avons démarré nos différentes tâches, notamment en changeant des affectations.

2	**MISE EN PROD**					
2.1	Développer le back-end	Alban / Loïs	28/03/25	14/04/25	16	0 %
2.2	Développer le front-end	Carnegie / Mehdi	28/03/25	14/04/25	16	0 %
2.3	Mettre en œuvre les plans de test	Carnegie / Loïs	14/04/25	21/04/25	7	0 %
2.4	Dockerisation	Alban / Mehdi	14/04/25	21/04/25	7	0 %
2.5	Superviser le fonctionnement	Mehdi / Loïs	14/04/25	28/04/25	14	0 %
2.6	Corriger les dysfonctionnements identifiés	Équipe entière	14/04/25	28/04/25	14	0 %
2.7	Commencer le rapport + les livrables	Équipe entière	14/04/25	21/04/25	7	0 %
2.8	Réaliser les évolutions fonctionnelles (améliorations post-feedback)	Équipe entière	21/04/25	28/04/25	7	0 %

Modélisation des données

Au début du projet nous avons pu réaliser un diagramme UML pour nous aider à mieux réfléchir et centraliser les idées tirées du sujet (Ici nous parlons globalement pas seulement sur la partie 1&2).

Ainsi qu'un modèle pour ensuite créé la base de données.
Modèle conceptuel de donnée (MCD) : S


Utilisateur
id : int
firebase_uid : str
email : str
nom : str
prenom : str
role_id : FK
is_active : bool
is_staff : bool
created_at : datetime
updated_at : datetime
deleted_at : datetime
1,N
1,1
1,N
0,N
possède
scan
écrit
Saved
id : int
user_id : FK
animal_id : FK
saved_at : datetime
0,N
1,N
1,1
Role
id : int
libelle : str
Log
id : int
user_id : FK
animal_id : FK
image_path : str
latitude : float
longitude : float
scanned_at : datetime
created_at : datetime
deleted_at : datetime
Article
id : int
titre : str
description : text
contenu : text
image : file
date_create : datetime
auteur_id : FK
categorie : str
is_active : bool
slug : str
1,N
1,N
concerne
1,N
Animal
id : int
nom_commun : str
nom_scientifique : str
espece : str
description : text
region : str
habitat : str
fun_fact : text
taille_min_cm : int
taille_max_cm : int
image : file
created_at : datetime
updated_at : datetime
deleted_at : datetime
1,N
associe
1,N
AnimalImage
id : int
animal_id : FK
path : str
description : str
order_index : int
is_thumbnail : bool
image : file
created_at : datetime
updated_at : datetime

BACK-END avec Django

Le développement du back-end de l'application WildLens a été réalisé avec une approche modulaire, sécurisée et évolutive. Il repose sur le framework Django, enrichi de Django REST Framework (DRF) pour la création d'API RESTful, et a été entièrement conteneurisé avec Docker afin de garantir une portabilité maximale.

Objectifs et rôle du back-end

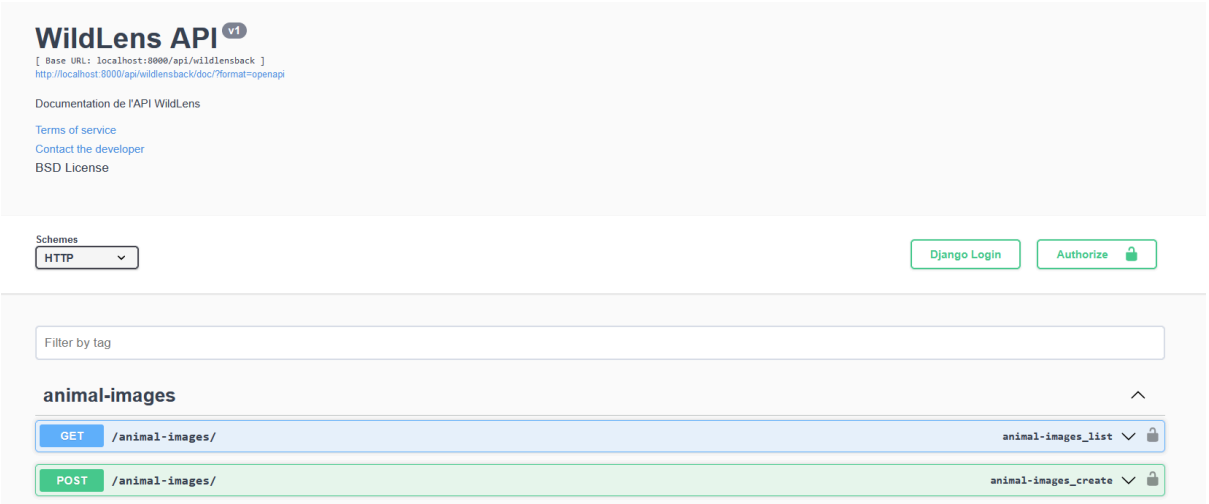
Le rôle principal du back-end est de faire le lien entre trois composants critiques :

- le front-end (interface utilisateur),
- la base de données PostgreSQL,
- et le modèle d'intelligence artificielle chargé de la classification des empreintes.

Il assure également le stockage des identifications réalisées par les utilisateurs, ainsi que l'exposition d'une API sécurisée consommable à distance.

Choix techniques et architecture

Élément	Choix effectué	Justification principale
Framework principal	Django + Django REST Framework	Productivité élevée, sécurité intégrée, API robuste et maintenable
Langage	Python	Compatibilité directe avec les bibliothèques IA, forte communauté
Base de données	PostgreSQL	Requêtes complexes, scalabilité, fiabilité
Conteneurisation	Docker	Portabilité, reproductibilité entre les environnements
Documentation	Swagger (via drf-yasg)	Génération automatique de la doc API à partir des vues DRF



Méthodologie et développement

Nous avons adopté une méthodologie agile inspirée du Kanban, avec un découpage clair des tâches backend :

- Création des modèles de données et de l'ORM Django.
- Définition des sérializers pour la conversion JSON ↔ objets Python.
- Construction des viewsets DRF et des routes d'API REST.
- Liaison entre les identifications, les métadonnées espèces, et les utilisateurs.
- Intégration du modèle IA via une route spécifique qui déclenche la classification.
- Création de tests unitaires et documentation Swagger.

Chaque étape a été documentée dans le README du dépôt backend, pour faciliter la prise en main par de futurs contributeurs.

Conteneurisation (Docker)

Le back-end est encapsulé dans un conteneur Docker à l'aide d'un Dockerfile personnalisé et orchestré via Docker Compose. Cette démarche permet :

- Une installation rapide sur n’importe quelle machine.
- Une séparation claire entre les services (API, BDD, modèle IA).
- Une cohérence des environnements de dev/test/prod.

Accessibilité & API publique

Même si l’accessibilité numérique concerne surtout le front-end, notre API respecte certains principes :

- **Uniformité des réponses**, gestion centralisée des erreurs.
- **Format JSON clair**, pour une exploitation par des interfaces accessibles.
- **Endpoints documentés et testables via Swagger**, facilitant l'intégration front.

Le back-end respecte également les **bonnes pratiques de sécurité et RGPD**, en ne stockant que les données nécessaires et en anonymisant les traces.

Le back-end est actuellement opérationnel et intégré avec le front-end mobile et web.

FRONT-END avec Flutter

1. Objectifs fonctionnels de l'application

Permettre à l'utilisateur de prendre une photo d'empreinte animale ou d'en choisir une dans sa galerie.
Envoyer cette image à une IA hébergée sur un backend pour analyse et identification.
Afficher les détails de l'animal reconnu avec une image, une description, des infos biologiques, un fun fact, etc.
Sauvegarder localement l'historique des analyses pour y accéder hors connexion.
Afficher des articles éducatifs, des animaux populaires, et des catégories.
Navigation fluide entre les pages, y compris retour au précédent état.

2. Design de l'application et maquettage initial

Avant même de commencer le développement, tout le design de l'application a été entièrement conçu sur Figma, aussi bien pour l'application mobile que la version web. Cette phase a permis de poser une vision claire et précise de l'expérience utilisateur, du parcours de navigation, ainsi que du positionnement visuel de chaque élément (boutons, cartes, textes, couleurs, images, etc.). Grâce à cette base graphique solide, le développement de l'interface utilisateur sous Flutter a pu être effectué en respectant fidèlement le rendu imaginé. Nous nous sommes concentrés sur le design de la version mobile en premier lieu, car l'application est destinée à être principalement utilisée sur téléphone lors de sorties en nature, là où les utilisateurs prennent directement des photos d'empreintes.

Objectifs du design initial :

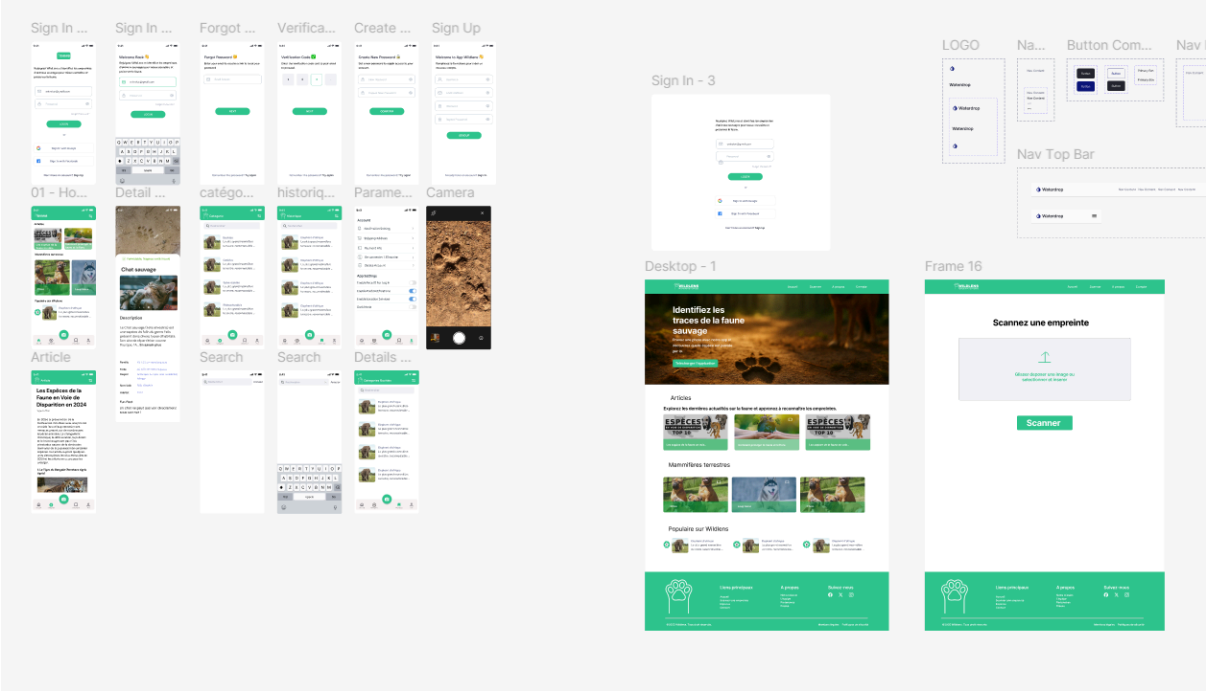
Anticiper les contraintes d'ergonomie sur web et mobile.
Définir une charte visuelle cohérente et attractive.
Optimiser l'agencement des éléments pour chaque plateforme (responsive design).
Accélérer le développement Flutter en disposant d'une maquette interactive.

Contenu du design :

Page d'accueil avec section articles, animaux populaires, et mammifères en vedette.
Page de caméra avec superposition du visuel et actions contextuelles.
Galerie avec grille d'images, sélection, et analyse manuelle.
Fiche d'animal avec image, description, infos clés, fun fact et navigation fluide.
Pages d'authentification : inscription, connexion, mot de passe oublié.

Paramètres utilisateurs avec dark mode, bouton de déconnexion, version et crédits.
Pages catégories, historique, détails d'article, etc.
Chaque écran a été pensé pour s'adapter aux différentes tailles d'écran, avec des composants réutilisables, une hiérarchie claire et un rendu visuel harmonisé. Le respect rigoureux de ces maquettes tout au long du développement a permis d'assurer la cohérence et la qualité visuelle de l'application finale, tant sur smartphone que sur navigateur.

Le lien du design figma:
https://www.figma.com/design/BhHh359rq7pg4Radq4KoaB/Mobile-App-Wildlens-Empreinte?node-id=0-1&p=f&t=VYEUKCbgmkRbsF7H-0



3. Fonctionnalités mises en œuvre

L'application WildLens offre un ensemble riche de fonctionnalités réparties en modules logiques. Elle allie ergonomie, rapidité d'accès à l'information, expérience utilisateur fluide et support de l'analyse IA. Voici un aperçu complet :

3.1. Détection d'empreintes animales (fonction principale)

Prise de photo via l'appareil photo.
Choix d'une image dans la galerie locale.
Analyse IA de l'empreinte pour identifier l'animal.

Envoi de l’image au backend via un MultipartRequest POST.
Récupération automatique des données de l’animal identifié.
Affichage complet de la fiche animal (infos, image, nom, description, fun fact).
Message visuel “Espèce trouvée” lorsque la détection IA est réussie.

3.2. Gestion des données et persistance

Mise en cache locale pour fonctionnement hors ligne :
Articles, animaux, et catégories sauvegardés dans shared_preferences.
Lecture automatique depuis le cache si le serveur n’est pas accessible.
Historique d’analyses IA :
Chaque animal identifié est sauvegardé localement.
Page dédiée (HistoryPage) pour retrouver les empreintes précédemment analysées.

3.3. Gestion de la galerie utilisateur

Affichage de toutes les images stockées localement (prises ou importées).
Ajout d’image depuis le téléphone.
Sélection visuelle de l’image active.
Lancement de l’analyse IA depuis cette interface.
Copie automatique des images importées dans un dossier dédié (/pictures).

3.4. Fiches d’animaux (standard et IA)

Présentation d’un animal complet avec :
Son nom
Une description
Une image principale
Une section “fun fact”
Des informations complémentaires :

- Famille
- Taille
- Région
- Nom scientifique
- Habitat

Affichage différent selon le contexte :

- Standard : fiche pleine page
- IA : fiche semi-couverte avec image en fond

3.5. Navigation par catégories

Page Catégories listant les espèces détectées par famille (ex : “Canidae”).
Chaque catégorie affiche :

- Un nom
- Une image associée automatiquement depuis la base de données

Appui sur une catégorie → liste des animaux correspondants.
Barre de recherche intégrée sur la page.

3.6. Section articles

Affichage de tous les articles récents.
Les articles sont dynamiques et récupérés via API.
Mise en cache locale pour lecture hors ligne.
Détail article : image, titre, date, contenu.

3.7. Page d’accueil dynamique

Mammifères terrestres mis en avant (6 sélectionnés manuellement).
3 animaux populaires (zone dédiée).
Liste d’articles récents avec image + titre.
Navigation intuitive par tap sur les cards.
Design fluide, responsive, visuellement cohérent.

3.8. Gestion des utilisateurs

Fichiers de base déjà présents : login_page.dart, register_page.dart, forgot_password.dart, etc.
Intégration du système de token prévue via AuthService.
Objectif : permettre à l’utilisateur de se connecter et garder ses données synchronisées.

3.9. Comportements intelligents

Envoi conditionnel de la photo seulement quand l’utilisateur valide (“Analyser”).
Sauvegarde locale automatique des images capturées.
Comportement de retour fluide avec Navigator.pop(context) cohérent partout.
Bouton retour visible même sur l’écran IA semi-transparent.

3.10. Paramètres & accessibilité

Page settings.dart avec :

- Un bouton de connexion/déconnexion dynamique (basé sur l'état utilisateur).
- Un switch Dark Mode pour basculer l'interface (avec état persistant).

Préparation au respect des normes d'accessibilité WCAG 2.1 niveau A/AA :

- Boutons avec labels compréhensibles.
- Contrastes visuels suffisants.
- Composants interactifs accessibles à la navigation clavier ou vocale.
- Exemple : le bouton de mode sombre respecte les bonnes pratiques d'accessibilité (focus, état clair, indication visuelle...).

3.11. Widgets personnalisés réutilisables

CustomAppBar : barre d'en-tête standardisée.
AnimalListCard, PopularItem, CategoriesCard, ArticleCard : composants visuels modulaires.
SearchBarWidget : barre de recherche utilisée dans history.dart et category_card.dart.

3.12 Gestion des permissions

Demande dynamique des permissions nécessaires :

- Accès à la caméra (Permission.camera)
- Accès aux photos / galerie (Permission.photos)

Compatibilité assurée pour Android API 33+ (avec fallback pour anciennes versions).
Permissions correctement déclarées dans le AndroidManifest.xml.

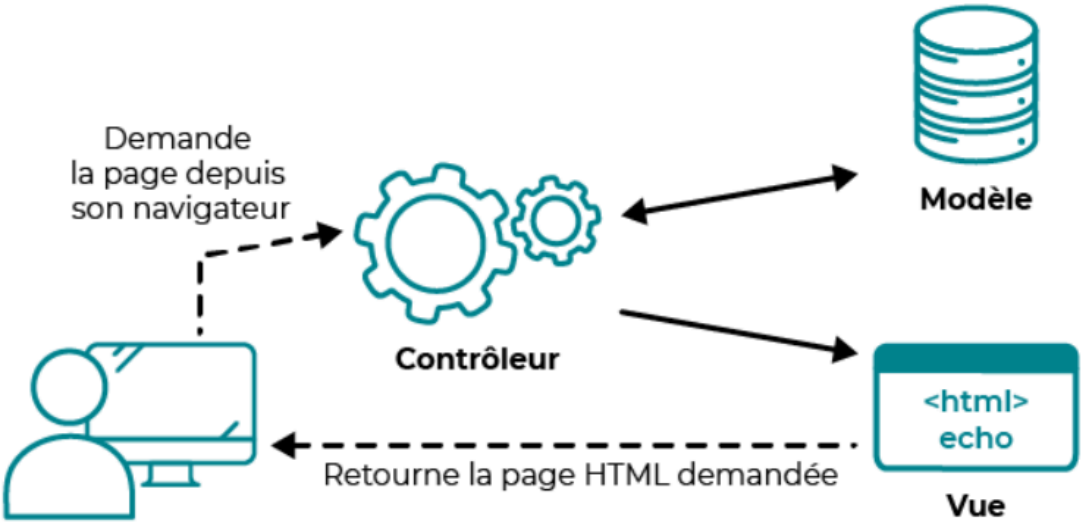
4. Composants techniques et structure fonctionnelle

Cette section présente en détail l'architecture logique de l'application WildLens, ainsi que la liste complète des composants implémentés : pages, modèles, services, widgets, gestion de l'état et de la navigation.

4.1 Architecture logique de l'application

L'application WildLens est structurée autour d'un modèle MVC simplifié, parfaitement adapté aux projets Flutter. Cette structure garantit une séparation claire entre les données, la logique métier et l'interface utilisateur.

- Modèles (Model) : Représentent les entités de données manipulées dans l'application (animaux, articles, catégories...).
- Vues (View) : Composées d'écrans (pages/) et de composants UI réutilisables (widgets/). Chaque vue est responsable de l'affichage et de l'interaction.
- Contrôleurs/Services : Gèrent la logique métier, les appels API et la gestion locale (dans services/).



4.2 Arborescence technique principale

```
lib/
├── auth/                    # Pages de gestion des utilisateurs
│   ├── login_page.dart
│   ├── register_page.dart
│   ├── forgot_password.dart
│   ├── verify_code.dart
│   └── new_password.dart
│
├── models/                  # Modèles de données
│   ├── animal_model.dart
│   ├── article_model.dart
│   └── category_model.dart
│
├── pages/                   # Toutes les pages principales de l'application
│   ├── home.dart
│   ├── category_card.dart
```

```
│   ├── animals_by_category.dart
│   ├── animal_details.dart
│   ├── gallery.dart
│   ├── camera.dart
│   ├── article_details.dart
│   ├── history.dart
│   └── settings.dart
│
├── services/                # Logique métier et intégration API
│   ├── api/
│   │   └── api.dart
│   ├── animal_service.dart
│   ├── article_service.dart
│   ├── category_service.dart
│   ├── analyze_service.dart
│   └── history_storage.dart
│
├── widgets/                 # Composants UI réutilisables
│   ├── custom_app_bar.dart
│   ├── animal_list_card.dart
│   ├── article_card.dart
│   ├── categories_card.dart
│   ├── popular_item.dart
│   └── search_bar_widget.dart
│
├── main.dart                # Fichier principal de l'application
```

4.3 Pages principales

HomePage : présente les articles, les animaux mis en avant et les populaires.

CameraPage : prend des photos à analyser, affiche un aperçu, lance l'analyse IA.
GalleryPage : permet de choisir une image depuis le téléphone, de l'analyser, et de visualiser les photos stockées localement.
HistoryPage : liste des animaux identifiés par l'utilisateur, stockés localement.
AnimalDetailsPage : affiche les détails d'un animal (design différent si issu d'une analyse IA).
ArticleDetailsPage : présente le contenu d'un article complet.
Login/Register Pages : interfaces de gestion de compte utilisateur.

4.4 Services métiers

AnimalService : charge les animaux et images associées, filtre par nom ou ID.
ArticleService : charge les articles avec fallback locales via shared_preferences.
CategoryService : extrait dynamiquement les catégories des animaux existants.
AnalyzeService : envoie une image au serveur et retourne un Animal reconnu par l'IA.
HistoryStorage : stocke l'historique d'animaux analysés localement (format JSON).

4.5 Modèles utilisés

Animal : nom, image, description, espèce, taille, région, habitat, fun fact...
Article : titre, contenu, image, date de création...
Category : id, titre, description, image.
Chaque modèle dispose d'un constructeur fromJson et d'un toJson pour la sérialisation locale.

4.6 Widgets personnalisés

ArticleCard : carte pour un article dans le carrousel horizontal.
AnimalListCard : carte d'animal en liste (utilisée pour HistoryPage, CategoriesPage...).
CategoriesCard : carte visuelle pour une espèce mise en avant.
PopularItem : présentation détaillée pour la section "Populaire sur WildLens".
SearchBarWidget : champ de recherche réutilisable dans Categories et History.
CustomAppBar : barre d'en-tête personnalisée avec le titre.

4.7 Pages authentifications

login_page.dart : interface de connexion des utilisateurs.
register_page.dart : interface d'inscription avec vérification des champs.
forgot_password.dart : permet à l'utilisateur de demander la réinitialisation de son mot de passe.
verify_code.dart : page pour entrer le code reçu par e-mail.
new_password.dart : définition du nouveau mot de passe après vérification du code.

4.8 Navigation et transition entre les pages

Navigation entre les pages gérée avec Navigator.push standard.
Les boutons de retour sont présents sur toutes les pages, même après analyse IA.
Comportement adapté après action utilisateur : retour vers la page précédente ou redirection vers la page de résultat (AnimalDetailsPage)

5. Problèmes rencontrés et solutions appliquées

Le développement de l'application WildLens a permis de consolider des compétences techniques en résolvant divers problèmes concrets liés à l'UI, aux permissions Android, à l'intégration backend ou encore à la compatibilité multiplateforme.

5.1. Problème d'affichage (overflow / SizedBox)

Contexte :
Dès le début du développement, un overflow visuel bloquait le rendu de certaines pages, notamment dans les interfaces ListView, SizedBox ou GridView.
Cause :

- Utilisation de SizedBox trop grands sans adaptation à la taille de l'écran.
- Conteneurs rigides dans des environnements scrollables.

Solution appliquée :

- Révision de toute la structure de certaines pages avec un Flexible, Expanded ou MediaQuery pour adapter les tailles dynamiquement.
- Ajout de SingleChildScrollView ou ListView pour éviter les débordements.

5.2. Permissions refusées (caméra, galerie, stockage)

Symptômes :

- Message : "Permission de stockage refusée" ou impossibilité d'accéder à la caméra ou à la galerie.

Cause :

- Mauvaise gestion des permissions selon les versions Android.
- Permission.storage devenu obsolète depuis Android API 33+.
- Absence de configuration dans le AndroidManifest.xml.

Solutions :

- Utilisation des permissions modernes :

dart
CopierModifier
```
await Permission.photos.request(); // à la place de Permission.storage
```

- Ajout dans le fichier AndroidManifest.xml :

xml
CopierModifier
```
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES"/>
<uses-permission android:name="android.permission.CAMERA"/>
```

· Gestion dynamique des permissions avec permission_handler.

5.3. Problème de compatibilité backend (analyse IA – erreur 404)

Problème :

- L'envoi d'image à l'IA déclenchait l'erreur :

pgsql
CopierModifier
Erreur IA ou animal non reconnu : Exception analyse IA : 404

Cause :

- Mauvais endpoint utilisé (identify/ non reconnu au début).
- ture de réponse non conforme (nom_commun au lieu de name).

Solution :

- Consultation de la documentation Swagger du backend.
- Adaptation du analyze_service.dart :

dart
CopierModifier
final url = Uri.parse('${ApiService.baseUrl}wildlensback/animal/identify/');

Traitement correct de la réponse avec le bon modèle Animal.

5.4. Problème de chargement d'image d'animaux

Erreur :

yaml
CopierModifier
NetworkImageLoadException – 404: default_animal.jpg introuvable

Cause :

- Endpoint d'image incorrect ou image principale non définie dans les données de l'animal.
- Mauvaise logique de récupération dans animal_service.dart.

Solution :

- Nouvelle stratégie d'association dynamique entre animal.id et l'image animal-images/ avec is_thumbnail = true.
- Refactor du Animal.fromJson() pour inclure cette logique proprement.

5.5. Problème de compatibilité multiplateforme (web/desktop)

Contexte :

- Le projet est multiplateforme (Android, iOS, Web, Desktop)

Risques :

- Incompatibilité de certains plugins (camera, path_provider, etc.) avec le web.
- Complexité de gestion des permissions ou du FileSystem.

Décision :

- Priorité au bon fonctionnement sur Android.
- Le support Web/Desktop est structurellement prêt (grâce à Flutter) mais certaines fonctionnalités comme la caméra ou la galerie sont désactivées automatiquement si la plateforme ne le permet pas (kIsWeb, etc.).

5.6. Problème de cache non persistant ou mal chargé

Symptôme :
À chaque ouverture, les données étaient rechargées depuis le backend.
Cause :
Absence de cache local avec SharedPreferences.
Solution :
Ajout d'un système de cache local automatique pour :
- les animaux (animal_service.dart)
- les articles (article_service.dart)
- les catégories (category_service.dart)

Le cache est utilisé en fallback si la connexion Internet est indisponible.

5.7. Problème de navigation et comportement post-analyse IA

Contexte :
Après analyse IA, la page AnimalDetailsPage devrait s'afficher avec l'image en arrière-plan et un message spécial.
Dans les autres cas, l'affichage devrait être standard.
Problème :
L'image couvrait le contenu ou la page apparaissait coupée.
Solution :
Création d'un layout intelligent :
- DraggableScrollableSheet activé uniquement si une photoTaken est présente.
- Sinon, affichage classique avec SafeArea pour un rendu plein écran naturel.

Mise en oeuvre des plans de tests pour le déploiement

Plans de tests fonctionnels et structurels

Tests unitaires et d’intégration

Un ensemble de tests unitaires et d’intégration est mis en place pour garantir la stabilité et la fiabilité du projet. Ces tests sont exécutés automatiquement via la suite *pytest*, et la couverture de code est vérifiée grâce à l’outil *pytest-cov*. Tous les tests sont intégrés dans le pipeline CI/CD, avec une exécution systématique à chaque mise à jour du code. Les tests couvrent plusieurs fonctions critiques, telles que *remove_leakages()*, *list_files()*, et *get_top_classes()*. Les rapports de couverture sont générés au format .xml et sont stockés pour un suivi continu dans le système CI/CD via GitHub Actions (fichier *tests.yml*).

Tests de sécurité

Afin de tester la robustesse du modèle face aux attaques, la classe *ModelSecurityTester* est utilisée. Deux attaques classiques sont simulées : **FGSM** (*Fast Gradient Sign Method*) et **PGD** (*Projected Gradient Descent*), qui altèrent légèrement les entrées pour tromper le modèle. En complément, une injection de bruit Gaussien est effectuée pour mesurer l'impact des perturbations aléatoires sur les performances du modèle. Les principales métriques d’évaluation de ces tests sont l’accuracy propre (*acc_clean),* l’accuracy après attaque **FGSM** (*acc_fgsm*), ainsi que le targeted success rate, qui mesure la capacité du modèle à classer les échantillons malgré l'attaque. Ces tests sont implémentés dans le fichier *model_security_tester.py.*

Tests de non-régression

Pour garantir la stabilité des performances à travers les différentes versions du modèle, une méthode de test de régression a été implémentée dans le fichier *evaluator.py* sous la forme de la fonction *regression_test()*. Cette méthode compare les performances du modèle actuel avec un baseline enregistré (fichier *baseline.npz*). Les métriques évaluées incluent l’*accuracy* et la *balanced accuracy*, permettant de détecter toute dégradation des performances. Si une variation supérieure à un seuil de tolérance prédéfini de 1e-3 est détectée, un signal d'alerte est généré et un rollback automatique du modèle vers la version précédente est effectué. Cette approche permet de maintenir une continuité dans les performances et d'éviter toute dégradation significative du modèle lors de son évolution.

Analyse des résultats et corrections

Gestion des déséquilibres de classes

La gestion des déséquilibres de classes est en partie assurée par des techniques d'augmentation des données, intégrées dans la classe *DataAugmenter*. Les méthodes de *mixup* et *cutmix* génèrent des données synthétiques en combinant des images de classes différentes. Ces techniques visent à améliorer la diversité des données et équilibrer les classes. La méthode *mixup* combine les pixels de deux images pour créer de nouvelles images, mais elle peut altérer certaines zones importantes de l'image, ce qui pourrait affecter la qualité de la prise de décision. La méthode *cutmix*, quant à elle, découpe des parties d'images et les remplace par d'autres, ce qui peut également entraîner une perte d'informations pertinentes. Pour minimiser les risques, ces techniques sont appliquées avec des paramètres soigneusement ajustés pour ne pas perturber les zones critiques pour le modèle.

Les performances du modèle sont constamment suivies à l'aide de métriques telles que l'accuracy et la *balanced accuracy*, ainsi que de rapports détaillés générés via la fonction *generate_report()*. De plus, les outils *profile_data_pipeline()* et *profile_training()* mesurent les temps de traitement et d'entraînement, garantissant ainsi une gestion optimale du pipeline.

La méthode *_balanced_accuracy()* calcule la moyenne des accuracies par classe, permettant d'évaluer les performances du modèle sur des classes sous-représentées. Cette métrique, combinée au *classification_report*, fournit des informations détaillées sur l'impact du déséquilibre des classes sur les résultats du modèle.

Détection de fuites de données

La méthode *remove_leakage()* permet de détecter et de supprimer les fuites de données en comparant les empreintes numériques (*checksums*) des fichiers entre les jeux de données d'entraînement, de validation et de test. Cela garantit qu'aucune donnée ne soit accidentellement partagée entre ces ensembles, ce qui pourrait fausser les résultats du modèle. Cette approche est essentielle pour maintenir l'intégrité du processus de formation et de validation du modèle, assurant ainsi des évaluations fiables des performances.

Supervision de la solution IA via le monitorage

Outils de monitorage opérationnels

Surveillance des performances

Pour assurer un suivi dynamique des performances, le monitoring s'appuie sur deux axes complémentaires : l'analyse approfondie des métriques ainsi que l'optimisation des ressources vie le profiling.
La méthode *genrate_report()* de la classe *ModelEvaluator* produit un rapport détaillé précisant les métriques suivantes :

- ***Accuracy globale* et *balanced accuracy***, déjà évoqué précédemment
- ***Matrice de confusion***, permet de visualiser les erreurs par classes en identifiant les confusions fréquentes
- ***F1-score macro/micro***, évaluant la précision et le rappel pondérés, cruciale pour les jeux de données déséquilibrés
- ***Classification Report***, détaillant la précision, le rappel et le F1-score par classe, mettant en lumière des sous-performances spécifiques

Ces métriques sont exportés au format .npz via la méthode *export_results()* permettant une comparaison historique et une meilleurs intégration dans les dashboards.

D'un autre côté, les ressources sont optimisées via le profiling avec les fonctions *profile_data_pipeline()* et *profile_training()*. Ces dernières évaluent le temps d'exécution des pipelines données et d'entraînement en secondes par batch. Ceci permet d'identifier les goulots d'étranglement, d'ajuster la taille des batches en conséquence et/ou d'activer les optimisation Tensorflow (exemple : *tf.data.AUTOTUNE*).

Interprétabilité

La fonction *visualize_errors()* permet d'afficher des heatmaps superposées sur les images mal classifiées, mettant en évidence les zones spécifiques où le modèle commet des erreurs. Cette visualisation offre une compréhension détaillée des faiblesses du modèle en montrant visuellement les éléments qui influencent les erreurs de prédiction.

Les images originales sont comparées aux cartes de chaleur générées par la méthode *Grad-CAM* via *_plot_comparison()*. Les cartes de chaleur sont

redimensionnées avec *cv2.resize()* pour correspondre précisément aux dimensions des images d'origine. De plus, des titres explicites indiquant le "True label" et le "Predicted label" sont ajoutés pour contextualiser les visualisations et faciliter l'interprétation des résultats. Ce processus permet d'analyser de manière plus ciblée les erreurs du modèle et de guider les ajustements nécessaires.

Méthodologie adapté

Traçabilité historique via l'export .npz

Comme évoqué brièvement précédemment, une traçabilité historique est effectuée via l'export .npz. Pour cela, la méthode *export_results()* de la classe *ModelEvaluator* sauvegarde les métriques clés dans un format compressé et interopérable. Cela permet une comparaison visuelle rapide entre les versions (exemple : graphique d'évolution de l'accuracy sur 6 mois) ainsi qu'une identification des tendances. Lors d'une régression détectée, les données historiques aident à isoler la cause.

Intégration proactive au CI/CD

Une intégration proactive au CI/CD a également été mise en place au travers d'alertes GitHub Actions, ces dernières se déclenchent en cas de baisse de couverture de code (<80%) ainsi qu'en cas d'échec des tests de sécurité (ex : acc_fgsm < 50%).

Visualisation centralisée

Une évolution future possible de cette partie serait un script agrégeant les fichiers .npz en graphiques interactifs (exemple : courbe d'accuracy over time, heatmaps de confusion comparatives…). Les métriques pourraient également être injectées dans Prometheus ou Grafana via un *adapter* Python permettant une interface en temps réel via des interfaces dédiées.

Détection proactive d’anomalies

Pipeline CI/CD comme bouclier préventif

La construction d’une image Docker est intégrée dans le CI/CD garantissant l’isolation des dépendances. De plus, les tests unitaires, de sécurité et d’intégration sont effectués à chaque push et pull request. Tout cela est centralisé dans le fichier *tests.yml*.

Validation matérielle et résilience GPU

Des optimisations de mémoire sont également effectuées au travers de l’activation de *cuda_malloc_async* pour une allocation dynamique de la mémoire GPU réduisant les risques d’Out-of-Memory lors de l'entraînement du modèle. De surcroît, la précision mixte (*mixed_float16*) divise par deux l’empreinte mémoire des tensors sans perte de précision significative et une vérification automatique de la compatibilité entre le GPU et CUDA est effectuée à chaque entraînement du modèle.

Correction des dysfonctionnements

Stratégies de correction cohérentes

Une gestion du déséquilibre des classes est partiellement prise en charge dans le pipeline de données via des techniques d'augmentation des données, intégrées dans la classe *DataAugmentor*. Une augmentation préalable de ces dernières est effectuée au travers des techniques de *mixup* et *cutmix*. Ces méthodes créent des images synthétiques en combinant des images de classes différentes afin d’augmenter la diversité de données et équilibrer les classes. Le *mixup* crée de nouvelles images en combinant les pixels de deux images. Cette approche présente un risque : les zones pertinentes d’une image pourraient être modifiées ce qui dégraderait leur qualité. D’un autre côté, le *cutmix* découpe des parties d’images et les remplace par d’autres. Cette méthode peut également entraîner la perte d’informations pertinentes.

Pour minimiser les risques, ces techniques sont utilisées avec des paramètres contrôlés (alpha=0.2) limitant les perturbations importantes des zones essentielles à la prise de décisions du modèle.

Optimisations techniques

L'implémentation de *cuda_malloc_async* (via la variable d'environnement TF_GPU_ALLOCATOR) permet une allocation dynamique de la mémoire GPU, réduisant les risques de Out-of-Memory (OOM) lors de l'entraînement de modèles volumineux comme ViT-large. Cette approche est particulièrement critique pour les architectures nécessitant des batch sizes importants (ex : 64+).

L'activation de *mixed_float16* (fichier *main.py*) divise par deux l'empreinte mémoire des tensors sans compromettre la précision des gradients, grâce à la conservation des poids en float32. Combinée à des gradient clipping (global_clipnorm=2.0 dans *training_pipeline.py*), cette technique permet d'entraîner des modèles complexes sur des GPUs grand public (ex : NVIDIA RTX 3090).

Amélioration continue et évolutions fonctionnelles

Fine-tuning dynamique

Dans le cadre de l'amélioration continue, un fine tuning a été mis en place afin d'adapter et d'affiner le modèle en fonction de nouvelles données ou d'ajustements nécessaires. Ce processus active progressivement certaines couches du modèle (*TrainingPipeline.unfreeze_layers()*) afin de permettre une adaptation précise du modèle en conservant les connaissances acquises lors de l'entraînement initial. L'entraînement doit être subtil lors de cette phase (LR * 0.1) afin de ne pas perturber les poids déjà appris.

Mécanismes d'adaptation future

A l'avenir, des scripts simulant des montées en charge pourraient être mis en place afin d'évaluer la scalabilité de l'inférence.

De plus, des validations A/B pourraient être effectuées afin de comparer en temps réel de deux versions de modèle via des endpoints dédiés mesurant l'impact des mises à jour sur les KPIs.

Optimisations post-déploiement

Les heatmaps générés par la fonction *generate_gradcam()* sont sauvegardés avec les métriques facilitant l'analyse rétrospective des erreurs.

DÉPLOIEMENT

Déploiement de la solution

Le déploiement de notre application WildLens a été conçu pour assurer une mise en production automatisé, reproductible et continue, en tenant compte des contraintes de portabilité et de scalabilité. Pour ce faire, nous avons structuré notre pipeline de déploiement autour de Docker, GitLab CI/CD et Railway.

1. Conteneurisation avec Docker

Notre back-end à était conteneurisé avec docker

Grâce à **Docker Compose**, nous avons pu orchestrer ces services de manière unifiée, facilitant les tests en local ainsi que les déploiements en pré-production ou production. Notre service dispose d'un Dockerfile propre, garantissant une cohérence de l'environnement entre les postes des développeurs et les environnements de production.

2. Intégration Continue (CI) avec GitLab

Pour le versionning, le suivi d'avancement, et l'automatisation du déploiement, nous avons utilisé GitLab. La plateforme nous a permis :

- De gérer les branches de développement via Git
- D'activer l'intégration continue (CI) pour valider automatiquement le code et assurer sa conformité
- De préparer la mise en place future de pipelines CI/CD pour le déploiement continu

Même si la pipeline GitLab CI/CD complète n'a pas encore été finalisée, nous avons structuré notre projet pour permettre une intégration facile des étapes de build, test et déploiement automatisé, notamment grâce à la conteneurisation.

3. Déploiement Continu (CD) avec Railway

Pour l'hébergement cloud, nous avons choisi **Railway.app**, une solution PaaS moderne et simple à connecter à GitLab.

Avantages de Railway :

- Support natif de **PostgreSQL**, ce qui simplifie la gestion de notre base.
- Déploiement automatisé à chaque push sur main.
- Monitoring intégré, logs consultables, redémarrage automatique des services.
- Déploiement de l'API Django en conteneur Docker (Dockerfile personnalisé).

Railway nous permet donc d'assurer la disponibilité de notre API et de notre base de données 24/7, avec une scalabilité immédiate si besoin.

Perspective :

Axes d'amélioration :

- Grafana : Utiliser un outil de visualisation plus avancé et directement après l'insertion en BDD.
- Scrapping des CSV : Automatisation complète de la récupération des métadonnées.
- Mise en place de CI-CD pour simplifier et accélérer le déploiement. (Passer de GitHub à GitLab)
- Dockeriser l'ensemble pour gagner en efficacité et éviter les erreurs.
- 2FA

Conclusion:

Le projet **WildLens** a permis la création d'une solution complète mêlant intelligence artificielle, développement logiciel et déploiement cloud. Grâce à une architecture modulaire et conteneurisée, chaque composant a été conçu pour assurer cohérence, évolutivité et performance. L'intégration de tests automatisés, de pipelines CI/CD et d'outils de monitoring assure la robustesse et la maintenabilité de la solution. Enfin, ce projet ouvre la voie à des évolutions futures telles que la

visualisation en temps réel, la scalabilité dynamique du modèle ou encore l’intégration d’analyses A/B pour valider les mises à jour en production.

Sources

Images :

https://www.inaturalist.org/
https://naturetracking.com/

Métadonnées :

https://www.kaggle.com/datasets/iamsouravbanerjee/animal-information-dataset?resource=download
https://www.diconimoz.com/animaux-fiches/mammiferes/liste/
https://www.instinct-animal.fr/liste-mammiferes/

Autres sources

Code de la propriété intellectuelle :
https://www.legifrance.gouv.fr/codes/article_lc/LEGIARTI000048603495
Débogage / aide au développement :
https://copilot.microsoft.com/
Modification et création d'un CSV avec erreur :
https://chatgpt.com/
AI utilisés pour la complétion du CSV :
https://mistral.ai/fr

Glossaire

ETL – *Extract Transform Load* : processus d'extraction, transformation et chargement de données.
BDD – *Base de Données* : système permettant de stocker, organiser et interroger des données.
CSV – *Comma-Separated Values* : format de fichier texte pour stocker des données tabulaires.
Parquet – Format de fichier optimisé pour le stockage et l'analyse de données volumineuses (utilisé notamment avec Spark).
API – *Application Programming Interface* : interface logicielle permettant à deux applications de communiquer entre elles (défini par la CNIL comme un connecteur de services).
IA – *Intelligence Artificielle* : discipline informatique visant à simuler l'intelligence humaine (ex : classification d'empreintes).
DRF – *Django REST Framework* : extension de Django pour construire facilement des API RESTful.
ORM – *Object-Relational Mapping* : technique qui permet de manipuler une BDD avec des objets dans le code (via Django notamment).
Docker – Plateforme de conteneurisation permettant de regrouper une application avec toutes ses dépendances.
CI/CD – *Continuous Integration / Continuous Deployment* : pratiques de développement automatisant les tests et le déploiement du code.
GitLab – Plateforme de versionnage de code, avec outils intégrés pour CI/CD.
Railway – Plateforme cloud facilitant le déploiement d'applications, compatible avec Docker.
Swagger – Outil de documentation interactive pour APIs RESTful, utilisé avec DRF pour générer des docs à partir du code.
pytest – Framework de tests pour Python, permettant d'écrire et exécuter des tests unitaires et d'intégration.
FGSM / PGD – Méthodes d'attaques adversariales visant à tester la robustesse des modèles IA.
Grad-CAM – Méthode de visualisation permettant de comprendre les décisions d'un modèle en surlignant les zones influentes d'une image.
Heatmap – Carte de chaleur représentant graphiquement les zones d'intérêt ou d'erreur dans une image.
Balanced Accuracy – Moyenne des taux de précision sur chaque classe, utile en cas de déséquilibre des données.
Mixup / Cutmix – Méthodes d'augmentation de données en IA pour générer des exemples synthétiques.

Dockerfile – Fichier de configuration indiquant comment construire un conteneur Docker.
Docker Compose – Outil permettant d'orchestrer plusieurs conteneurs (ex : backend + BDD).
npz – Format compressé de NumPy pour stocker des tableaux de données, utile pour archiver des métriques IA.
MVC – (Model-View-Controller ou *Modèle-Vue-Contrôleur*) est un modèle dans la conception de logiciels. Il met l'accent sur la séparation entre la logique métier et l'affichage du logiciel. Cette «séparation des préoccupations» permet une meilleure répartition du travail et une maintenance améliorée.